

UNIT -1

Level A Easy Questions:

Q1: How would you create a hyperlink (a link) to another website in HTML?

You can create a hyperlink using the <a> tag in HTML.

Example:

```
<a href="https://www.example.com">Visit Example</a>
```

Here, href is the attribute that specifies the URL of the page the link points to.

Q2: Explain the difference between inline elements and block elements in HTML.

- **Inline Elements:** Do not start on a new line and occupy only as much width as necessary. Example: , <a>.
 - **Block Elements:** Start on a new line and take up the full width available. Example: <div>, <p>.
-

Q3: Describe the use of the <blockquote> tag in HTML.

The <blockquote> tag is used to define a section that is quoted from another source.

Example:

```
<blockquote>
```

```
"The journey of a thousand miles begins with one step."
```

```
</blockquote>
```

Q4: How do you add attributes to HTML elements? Provide an example.

Attributes provide additional information about HTML elements and are always written in the opening tag.

Example:

```

```

Q5: Create one webpage and show the use of the style attribute.

```
<!DOCTYPE html>
<html>
<head>
  <title>Styled Page</title>
</head>
<body>
  <p style="color: blue; font-size: 18px;">This is a blue paragraph with custom styling.</p>
</body>
</html>
```

Q6: Can we write <H1> tag in the <head> tag? Show with the help of an example.

No, the <h1> tag is meant to be used within the <body> tag, not the <head> tag. Using it in the <head> tag is invalid and will not render properly.

Example (Correct Use):

```
<!DOCTYPE html>
<html>
<head>
  <title>Invalid Example</title>
</head>
<body>
  <h1>Heading Inside Body</h1>
</body>
</html>
```

Q7: How can you change the color of text or background in HTML? Give an example.

```
<p style="color: red;">This text is red.</p>
<div style="background-color: yellow;">This div has a yellow background.</div>
```

Q8: How can you change the color of text or background in HTML? Give an example.

(Same as Q7; refer above.)

Q9: Create an HTML list containing three items. Two items should be ordered, and one unordered.

```
<ol>
  <li>Ordered Item 1</li>
  <li>Ordered Item 2</li>
</ol>
<ul>
  <li>Unordered Item</li>
</ul>
```

Q10: Construct an HTML table with two rows and three columns, containing sample data.

```
<table border="1">
  <tr>
    <th>Column 1</th>
    <th>Column 2</th>
    <th>Column 3</th>
  </tr>
  <tr>
    <td>Row 1, Col 1</td>
    <td>Row 1, Col 2</td>
    <td>Row 1, Col 3</td>
  </tr>
  <tr>
    <td>Row 2, Col 1</td>
    <td>Row 2, Col 2</td>
    <td>Row 2, Col 3</td>
  </tr>
</table>
```

Q11: List and briefly explain three common attributes you can use within an HTML <form> tag.

1. **action:** Specifies the URL where the form data should be sent.
2. **method:** Defines the HTTP method (GET or POST) used when sending form data.
3. **enctype:** Specifies the encoding type when submitting form data (e.g., multipart/form-data for file uploads).

Q12: How can you ensure that a user's input meets specific criteria in an HTML form before submission?

You can use attributes like required, maxlength, pattern, and type.

Example:

```
<input type="text" required pattern="[A-Za-z]{3,}" title="Enter at least 3 letters">
```

Q13: How can you specify colors in HTML using color names and hexadecimal values?

- **Using Color Names:**

```
<p style="color: red;">Red Text</p>
```

- **Using Hexadecimal Values:**

```
<p style="color: #FF0000;">Red Text</p>
```

Q14: Describe the purpose of the action and method attributes in an HTML form.

- **action:** Specifies the URL where the form data is submitted.
- **method:** Specifies the HTTP method (GET or POST) used for submitting the form.

Q15: How can you create radio buttons and checkboxes in a form?

Example: `<form>`

```
<label><input type="radio" name="option" value="1"> Option 1</label>
```

```
<label><input type="radio" name="option" value="2"> Option 2</label>
```

```
<br>
```

```
<label><input type="checkbox" name="check1"> Checkbox 1</label>
```

```
<label><input type="checkbox" name="check2"> Checkbox 2</label>
```

```
</form>
```

Q16: Provide an example of client-side form validation using HTML attributes.

```
<form>

  <input type="email" required placeholder="Enter your email">

  <input type="password" required minlength="6" placeholder="Enter at least 6
characters">

  <button type="submit">Submit</button>

</form>
```

Q17: Provide an example of using the Geolocation API in HTML.

```
<button onclick="getLocation()">Get Location</button>

<p id="location"></p>

<script>

  function getLocation() {
    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition((position) => {
        document.getElementById("location").innerHTML =
          `Latitude: ${position.coords.latitude}, Longitude: ${position.coords.longitude}`;
      });
    } else {
      document.getElementById("location").innerHTML = "Geolocation is not supported.";
    }
  }

</script>
```

Q18: Explain the process of form validation in HTML, including client-side and server-side validation.

- **Client-Side Validation:** Performed in the browser using HTML attributes or JavaScript. Ensures faster feedback and reduces server load.
- **Server-Side Validation:** Performed on the server to ensure data integrity and security, even if client-side validation is bypassed.

Q19: How can you include an audio element in an HTML document to play a sound file?

<audio controls>

<source src="audiofile.mp3" type="audio/mpeg">

Your browser does not support the audio element.

</audio>

Q20: Discuss the concept of HTML APIs. Provide an example of an API commonly used in web development.

- HTML APIs are interfaces that allow developers to use additional features in their web applications.
- **Example:** The Geolocation API lets you access a user's geographical location.

Refer to Q17 for an example implementation.

Q21: Explain the difference between an absolute URL and a relative URL when creating links in HTML. Additionally, describe the purpose of the element and how you would use it to display an image on a webpage.

1. **Absolute URL:** Specifies the full path to a resource, including the protocol, domain, and file path.
Example:

Visit Example

2. **Relative URL:** Specifies the resource path relative to the current document's location.
Example:

Visit Example

3. **Purpose of Element:** Used to embed images into a webpage.
Example:

Q22: What is the purpose of the HTML element? How does it differ from the element? Provide an example of when each one might be used in a webpage.

1. ** Element:** Displays text in bold without emphasizing its importance.
Example:

<p>This is a bold word.</p>

2. ** Element:** Displays text in bold but also indicates importance or urgency.

Example:

```
<p><strong>Warning:</strong> This action is irreversible.</p>
```

Q23: Name three commonly used input types in HTML forms and briefly describe their purposes. How can the required attribute be used to enhance form validation?

1. **Input Types:**

- **Text:** For single-line text input.
- **Email:** For email addresses with format validation.
- **Password:** For securely masking user input.

2. **Using required Attribute:** Ensures the field cannot be left empty before form submission.

Example:

```
<form>
```

```
<input type="text" required placeholder="Enter your name">
```

```
<input type="email" required placeholder="Enter your email">
```

```
<input type="password" required minlength="6" placeholder="Enter a password">
```

```
<button type="submit">Submit</button>
```

```
</form>
```

Q24: Describe the process of creating a basic HTML form. Explain the significance of the <form>, <input>, and <button> elements within the form structure.

1. **Steps to Create a Basic Form:**

- Use the <form> tag to wrap form elements.
- Add <input> elements for user data input.
- Include a <button> or <input type="submit"> to submit the form.

2. **Significance:**

- **<form>:** Defines the form structure and handles data submission.
- **<input>:** Collects user data.
- **<button>:** Triggers form submission.

Example:

```
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">
  <button type="submit">Submit</button>
</form>
```

Q25: Develop a complex HTML form for user registration, including form validation using HTML5 attributes and JavaScript.

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
  <style>
    .error { color: red; }
  </style>
</head>
<body>
  <form id="registrationForm">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required pattern="[A-Za-z\s]{3,}">
    <span class="error" id="nameError"></span><br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
    <span class="error" id="emailError"></span><br><br>

    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required minlength="6">
    <span class="error" id="passwordError"></span><br><br>
```

```
<label for="dob">Date of Birth:</label>

<input type="date" id="dob" name="dob" required>

<span class="error" id="dobError"></span><br><br>

<button type="submit">Register</button>

</form>

<script>

  document.getElementById("registrationForm").addEventListener("submit", function
(event) {

    let valid = true;

    const name = document.getElementById("name").value;

    if (!/^[A-Za-z\s]{3,}$/ .test(name)) {

      valid = false;

      document.getElementById("nameError").textContent = "Invalid name!";

    }

    if (!valid) {

      event.preventDefault();

    }

  });

</script>

</body>

</html>
```

Q26: Explain how to create a multimedia-rich webpage with images, videos, and audio in a grid-based gallery format.

1. HTML Structure:

```
<section>

<div>

  

  <audio controls><source src="audio.mp3"></audio>

  <video controls><source src="video.mp4"></video>

</div>

</section>
```

2. Techniques for Optimization:

- Use the loading="lazy" attribute for images and videos.
- Use responsive design techniques with CSS.

Q27: Explain the purpose of the alt attribute in HTML images and why it is important.

- The alt attribute provides alternative text for an image if it cannot be displayed.
- **Importance:**
 - Enhances accessibility for visually impaired users.
 - Improves SEO.

Example:

```

```

Q28: How can the href attribute in anchor tags be used to link to external websites and internal sections within the same webpage?

1. External Links:

```
<a href="https://www.example.com">Visit Example</a>
```

2. Internal Sections:

```
<a href="#section1">Go to Section 1</a>
```

```
<h2 id="section1">Section 1</h2>
```

Q29: Create an HTML document structure with specific elements.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>My First Web Page</title>

</head>

<body>

  <h1>Welcome to HTML Basics</h1>

  <p>HTML stands for Hypertext Markup Language.</p>

  <p>It is used to structure content on the web.</p>

</body>

</html>
```

Q30: Style the heading and paragraph, and create an unordered list.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Styled Web Page</title>

  <style>

    h1 {

      color: red;

      font-size: 24px;

    }

    p:nth-of-type(2) {

      font-style: italic;

    }

  </style>

</head>
```

```
<body>

  <h1>Welcome to HTML Basics</h1>

  <p>HTML stands for Hypertext Markup Language.</p>

  <p>It is used to structure content on the web.</p>

  <ul>

    <li>Apples</li>

    <li>Bananas</li>

    <li>Oranges</li>

  </ul>

</body>

</html>
```

Q31: Create an HTML form with labels and POST method.

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Form Example</title>

  </head>

  <body>

    <form method="POST" action="/submit">

      <label for="name">Name:</label>

      <input type="text" id="name" name="name" required><br><br>

      <label for="email">Email:</label>

      <input type="email" id="email" name="email" required><br><br>

      <button type="submit">Submit Form</button>
```

```
</form>
</body>
</html>
```

Q32: Create a navigation bar.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Navigation Bar</title>
</head>
<body>
  <nav>
    <a href="index.html">Home</a> |
    <a href="about.html">About</a> |
    <a href="contact.html">Contact</a>
  </nav>
</body>
</html>
```

Q33: Embed a video and an audio element.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Multimedia</title>
</head>
```

```
<body>

  <h2>Embedded Video</h2>

  <iframe width="560" height="315"
src="https://www.youtube.com/embed/dQw4w9WgXcQ" title="YouTube video"
frameborder="0" allowfullscreen></iframe>


  <h2>Embedded Audio</h2>

  <audio controls>

    <source src="audio-file.mp3" type="audio/mpeg">

    Your browser does not support the audio element.

  </audio>

</body>
</html>
```

Q34: HTML5 Input Types – Explanation and Examples

1. **Email Input:** Validates email format automatically.

Example:

```
<input type="email" required>
```

2. **Date Input:** Provides a date picker interface.

Example:

```
<input type="date" required>
```

Difference: Unlike standard text inputs, these types offer built-in validation and specific UI enhancements (e.g., email format checks, calendar pickers).

Q35: HTML Forms and Validation

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Form with Validation</title>
```

```
</head>
<body>
  <form>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required><br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required><br><br>

    <label>Gender:</label>
    <input type="radio" name="gender" value="Male"> Male
    <input type="radio" name="gender" value="Female"> Female<br><br>

    <label for="country">Country:</label>
    <select id="country" name="country" required>
      <option value="India">India</option>
      <option value="USA">USA</option>
      <option value="Canada">Canada</option>
    </select><br><br>

    <input type="checkbox" id="terms" required>
    <label for="terms">I agree to the terms and conditions</label><br><br>

    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

a) Difference Between Ordered and Unordered Lists

- **Ordered List ():** Displays items with a sequence (numbers or letters).

Example:

```
<ol>
  <li>First Item</li>
  <li>Second Item</li>
</ol>
```

- **Unordered List ():** Displays items with bullet points.

Example:

```
<ul>
  <li>Item One</li>
  <li>Item Two</li>
</ul>
```

b) HTML Table Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Table</title>
</head>
<body>
  <table border="1">
    <thead>
      <tr>
        <th>Name</th>
        <th>Age</th>
        <th>Country</th>
      </tr>
    </thead>
```

```
<tbody>
  <tr>
    <td>Alice</td>
    <td>25</td>
    <td>USA</td>
  </tr>
  <tr>
    <td>Bob</td>
    <td>30</td>
    <td>Canada</td>
  </tr>
  <tr>
    <td>Carol</td>
    <td>22</td>
    <td>UK</td>
  </tr>
</tbody>
</table>
</body>
</html>
```

Significance:

- <table>: Defines the table.
- <thead>: Contains header rows.
- <tbody>: Contains data rows.
- <tr>: Represents a table row.
- <th> and <td>: Define table headers and data cells.

UNIT-2

Level A - Easy Questions (2 Marks Each)

Q1. In how many ways can a CSS be integrated into a web page (types of CSS)?

CSS can be integrated into a web page in **three ways**:

1. **Inline CSS:** Styles are applied directly within an HTML element using the style attribute.
 2. **Internal CSS:** Styles are written within a <style> tag in the <head> section of the document.
 3. **External CSS:** Styles are placed in a separate .css file and linked to the HTML document.
-

Q2. What is the syntax to link the external style sheet?

```
<link rel="stylesheet" href="styles.css">
```

Q3. What is a hyperlink? Explain internal and external hyperlinks.

- **Hyperlink:** A hyperlink is a link that allows users to navigate from one webpage or resource to another. It is created using the <a> tag.
- **Internal Hyperlink:** Links to another section within the same website.
Example:

```
<a href="about.html">About Us</a>
```

- **External Hyperlink:** Links to a webpage on a different website.
Example:

```
<a href="https://www.example.com">Visit Example</a>
```

Q4. What are the different variations of CSS?

1. **Inline CSS**
 2. **Internal CSS**
 3. **External CSS**
 4. **CSS Preprocessors** (e.g., SASS, LESS)
-

Q5. What is a CSS selector? Write the names of 5 CSS selectors.

- A CSS selector is a pattern used to select HTML elements to style them.
 - **Examples of CSS selectors:**
 1. Universal Selector (*)
 2. Type Selector (div, p, etc.)
 3. Class Selector (.classname)
 4. ID Selector (#idname)
 5. Descendant Selector (div p)
-

Q6. What is the use of CSS Opacity?

- The opacity property in CSS sets the transparency level of an element.
- Values range from 0 (completely transparent) to 1 (completely opaque).
Example:

```
div {  
  opacity: 0.5; /* 50% transparent */  
}
```

Q7. Explain the universal selector.

- The universal selector (*) applies styles to **all elements** on a webpage.
Example:

```
* {  
  margin: 0;  
  padding: 0;  
}
```

Q8. What is CSS flexbox?

- **CSS Flexbox** (Flexible Box Layout) is a layout model that provides an efficient way to align and distribute space among items in a container, even when their sizes are dynamic.

```
display: flex;  
justify-content: center;
```

Q9. What is the difference between inline, embedded, and external style sheets?

- **Inline CSS:** Applied directly within an element using the style attribute.
 - **Embedded CSS:** Defined inside a <style> tag in the <head> section.
 - **External CSS:** Written in a separate .css file and linked using <link>.
-

Q10. What is CSS Box Model, and what are its elements?

- The **CSS Box Model** describes the structure of elements. It consists of:
 1. **Content:** The actual content.
 2. **Padding:** Space between the content and the border.
 3. **Border:** A boundary around the padding.
 4. **Margin:** Space outside the border.
-

Q11. What is the float property of CSS?

- The float property is used to position elements to the **left** or **right** of their container, allowing text or other content to flow around it.

Example:

```
img {  
  float: right;  
}
```

Q12. What is the purpose of the z-index, and how is it used?

- The z-index property specifies the **stacking order** of elements.
- Elements with higher z-index values appear in front of elements with lower values.

Example:

```
div {  
  z-index: 10;  
}
```

Q13. Explain the difference between visibility: hidden and display: none.

- **visibility: hidden:** Hides the element but keeps its space in the layout.
 - **display: none:** Hides the element and removes it from the layout.
-

Q14. Explain Min-height and Max-height property in CSS.

- **min-height:** Sets the minimum height of an element.
- **max-height:** Sets the maximum height of an element.

Example:

```
div {  
  min-height: 100px;  
  max-height: 300px;  
}
```

Q15. What does the 'a' in rgba mean?

- The **a in rgba** stands for **alpha**, which controls the transparency level of a color.
- Value ranges: 0 (completely transparent) to 1 (completely opaque).

Example:

```
background-color: rgba(255, 0, 0, 0.5); /* 50% transparent red */
```

Q16. What is CSS HSL Colors?

- HSL stands for **Hue, Saturation, and Lightness**.

Example:

```
background-color: hsl(120, 50%, 50%); /* Green */
```

Q17. What are the different CSS border properties?

1. border-width
 2. border-style
 3. border-color
 4. border-radius
 5. border
-

Q18. What is the difference between margin and padding?

- **Margin:** Space **outside** an element.
 - **Padding:** Space **inside** an element, between the content and the border.
-

Q19. What does the CSS float property do?

- Positions an element to the left or right of its container, allowing other content to flow around it.
-

Q20. How can we make our website responsive using CSS?

1. Use **media queries**.
2. Apply **flexbox** and **grid** layouts.
3. Use **relative units** like percentages and em.
4. Implement **responsive images**.
5. Apply **viewport meta tags**.

LEVEL B - INTERMEDIATE QUESTIONS (5 MARKS EACH)

Q21. What are CSS backgrounds? List the properties with Syntax.

CSS Backgrounds are used to define the appearance of an element's background.

Background properties in CSS:

1. **background-color:** Sets the background color.

`background-color: lightblue;`

2. **background-image:** Sets a background image.

`background-image: url('image.jpg');`

3. **background-repeat:** Controls whether the background image repeats.

`background-repeat: no-repeat;`

4. **background-position:** Defines the starting position of the background image.

`background-position: center;`

5. **background-size:** Specifies the size of the background image.

`background-size: cover;`

6. **background-attachment**: Determines if the background scrolls with the page.

background-attachment: fixed;

Q22. How can we format text in CSS? Write with Syntax.

CSS text formatting properties:

1. **color**: Sets the text color.

color: red;

2. **font-size**: Specifies the size of the text.

font-size: 20px;

3. **text-align**: Aligns text.

text-align: center;

4. **text-transform**: Changes the text case.

text-transform: uppercase;

5. **font-style**: Applies italic text.

font-style: italic;

6. **font-weight**: Adjusts the text weight (e.g., bold).

font-weight: bold;

Q23. Write JavaScript Program in HTML document to Print the Fibonacci Sequence.

```
<!DOCTYPE html>
<html>
<head>
  <title>Fibonacci Sequence</title>
</head>
<body>
  <h3>Fibonacci Sequence</h3>
  <script>
    function fibonacci(n) {
      let a = 0, b = 1, c;
```

```
document.write(a + " " + b + " ");  
for (let i = 2; i < n; i++) {  
    c = a + b;  
    document.write(c + " ");  
    a = b;  
    b = c;  
}  
}  
  
fibonacci(10); // Prints the first 10 Fibonacci numbers  
</script>  
</body>  
</html>
```

Q24. What are the key differences between Java and JavaScript?

Feature	Java	JavaScript
Type	Object-oriented language.	Scripting language.
Compilation	Compiled (bytecode).	Interpreted.
Platform	Runs in JVM.	Runs in web browsers.
Usage	Backend development.	Web development.
Syntax	Strongly typed.	Loosely typed.
Execution	Requires JDK/IDE.	Requires a browser/Node.js.

Q25. Explain the following JavaScript functions:**1. substring():**

- Extracts a portion of a string between two specified indices.
- Syntax:

```
string.substring(startIndex, endIndex);
```

- Example:

```
let str = "JavaScript";  
console.log(str.substring(0, 4)); // Output: "Java"
```

2. unshift():

- Adds one or more elements to the beginning of an array.
- Syntax:

```
array.unshift(element1, element2, ...);
```

- Example:

```
let arr = [2, 3];  
arr.unshift(1);  
console.log(arr); // Output: [1, 2, 3]
```

3. concat():

- Combines two or more arrays or strings.
- Syntax:

```
array1.concat(array2, array3, ...);
```

- Example:

```
let arr1 = [1, 2];  
let arr2 = [3, 4];  
console.log(arr1.concat(arr2)); // Output: [1, 2, 3, 4]
```

Q26. Write a function in JavaScript to find the greatest number among three numbers.

```
function findGreatest(a, b, c) {  
  if (a > b && a > c) {  
    return a;  
  } else if (b > a && b > c) {  
    return b;  
  } else {  
    return c;  
  }  
}
```

```
// Example usage
```

```
let greatest = findGreatest(10, 20, 15);
```

```
console.log("The greatest number is: " + greatest); // Output: 20
```

Q27. Explain the difference between break and continue with examples.

1. **break:** Exits the loop entirely.

Example:

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) break;  
  console.log(i); // Output: 0, 1, 2  
}
```

2. **continue:** Skips the current iteration and moves to the next one.

Example:

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) continue;  
  console.log(i); // Output: 0, 1, 2, 4  
}
```

Q28. Differentiate between prompt() and alert() methods.

Feature	prompt()	alert()
Purpose	To collect user input.	To display a message.
Syntax	prompt("Message", "Default value")	alert("Message")
Returns	User input as a string.	Undefined.
Example	let name = prompt("Enter your name:");	alert("Welcome!");

LEVEL C - DIFFICULT QUESTIONS (10 MARKS EACH)

Q29. What is the main difference between var, const, and let? Explain with a program.

1. var:

- Function-scoped.
- Can be re-declared and updated.
- Hoisted with undefined.

2. let:

- Block-scoped.
- Cannot be re-declared in the same scope but can be updated.
- Hoisted without initialization.

3. const:

- Block-scoped.
- Cannot be re-declared or updated.
- Must be initialized during declaration.

Example Code:

```
function scopeExample() {  
  var x = 10; // Function-scoped  
  if (true) {  
    var x = 20; // Re-declared  
    console.log("Inside block (var):", x); // Output: 20  
  }  
  console.log("Outside block (var):", x); // Output: 20  
  
  let y = 30; // Block-scoped  
  if (true) {  
    let y = 40; // New variable in block scope  
    console.log("Inside block (let):", y); // Output: 40  
  }  
  console.log("Outside block (let):", y); // Output: 30  
  
  const z = 50; // Block-scoped
```

```
    console.log("Constant value:", z); // Output: 50
  }
scopeExample();
```

Q31. Create an external style sheet “style.css” with the following specifications:

CSS Code (style.css):

```
body {
    background-color: aqua;
    color: navy;
}

h1 {
    text-decoration: underline;
    text-align: center;
}

p {
    text-indent: 60px;
    font-weight: normal;
}
```

HTML Code to Link the CSS:

html

Copy code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>External CSS Example</title>
```

```
<link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Welcome</h1>
  <p>This is an example paragraph demonstrating the linked external style sheet.</p>
</body>
</html>
```

Placement: The <link> tag should be placed inside the <head> section of the HTML document.

Q32. Difference between Inline-Block, Block, and Inline in CSS

1. **Block:** Takes up the full width of its container. Starts on a new line.

```
<div style="display: block;">This is block</div>
```

2. **Inline:** Only takes up as much width as its content. Doesn't start on a new line.

```
<span style="display: inline;">This is inline</span>
```

3. **Inline-Block:** Behaves like inline, but allows setting width and height.

```
<span style="display: inline-block; width: 100px; height: 50px; background: lightblue;">This
is inline-block</span>
```

Example:

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .block {
      display: block;
      background-color: lightblue;
    }
    .inline {
      display: inline;
      background-color: lightgreen;
```

```
}  
.inline-block {  
  display: inline-block;  
  width: 100px;  
  height: 50px;  
  background-color: lightcoral;  
}  
</style>  
</head>  
<body>  
  <div class="block">Block</div>  
  <span class="inline">Inline</span>  
  <span class="inline-block">Inline-Block</span>  
</body>  
</html>
```

Q33. Ways to Change Font Size of an Element

1. Using Inline CSS:

```
<p style="font-size: 20px;">This is inline CSS.</p>
```

2. Using Internal CSS:

```
<style>  
  p {  
    font-size: 22px;  
  }  
</style>  
<p>This is internal CSS.</p>
```

3. Using External CSS:

```
p {  
  font-size: 24px; }
```

4. Using JavaScript:

```
<script>
    document.getElementById("example").style.fontSize = "26px";
</script>
<p id="example">Font size changed with JavaScript.</p>
```

Q34. Create a Horizontal Navigation Menu using CSS

```
<!DOCTYPE html>
<html>
<head>
    <style>
        ul {
            list-style-type: none;
            margin: 0;
            padding: 0;
            overflow: hidden;
            background-color: #333;
        }

        li {
            float: left;
        }

        li a {
            display: block;
            color: white;
            text-align: center;
            padding: 14px 16px;
            text-decoration: none;
```

```
}

li a:hover {
    background-color: #111;
}

</style>
</head>
<body>
    <ul>
        <li><a href="#home">Home</a></li>
        <li><a href="#about">About</a></li>
        <li><a href="#services">Services</a></li>
        <li><a href="#contact">Contact</a></li>
    </ul>
</body>
</html>
```

Q35. Concept of Flexbox and Its Advantages

Flexbox: A CSS layout model that provides a way to align items along one axis (horizontal or vertical).

Advantages:

1. Simplifies alignment and distribution of elements.
2. Supports responsive design.
3. Eliminates the need for floats or positioning.

Example Code:

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .container {
```

```
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
background-color: lightblue;
}
.box {
width: 100px;
height: 100px;
margin: 10px;
background-color: coral;
}
</style>
</head>
<body>
<div class="container">
<div class="box"></div>
<div class="box"></div>
<div class="box"></div>
</div>
</body>
</html>
```

Q36. CSS Shorthand Properties

CSS shorthand properties allow combining multiple properties into one declaration.

Examples:

1. Margin:

margin: 10px 20px 30px 40px; /* top, right, bottom, left */

2. Padding:

```
padding: 10px; /* All sides */
```

3. **Font:**

```
font: italic bold 16px Arial, sans-serif;
```

4. **Background:**

```
background: lightblue url('image.jpg') no-repeat center center;
```

5. **Border:**

```
border: 1px solid black;
```

Advantages:

- Reduces code length.
- Improves readability.
- Simplifies maintenance.

UNIT-3

LEVEL A - EASY QUESTIONS (2 MARKS EACH)

Q1. List 5 features of JavaScript.

1. **Lightweight and Interpreted:** Runs directly in the browser without requiring compilation.
 2. **Dynamic Typing:** Variables do not need a type declaration.
 3. **Event-Driven Programming:** Handles events like clicks and keypresses.
 4. **Object-Oriented:** Supports objects and prototypal inheritance.
 5. **Versatile:** Works on the client-side, server-side, and as part of various frameworks.
-

Q2. Difference Between Client-Side and Server-Side JavaScript

Client-Side JavaScript

Runs in the user's browser.

Used for UI interactivity.

Server-Side JavaScript

Runs on the server.

Used for backend logic.

Client-Side JavaScript**Server-Side JavaScript**

Limited access to the system. Full access to server resources.

Q3. Types of Variables in JavaScript

1. **var**: Function-scoped, can be re-declared.
 2. **let**: Block-scoped, cannot be re-declared in the same scope.
 3. **const**: Block-scoped, cannot be updated or re-declared.
-

Q4. Function of document.write() Method

The document.write() method writes content directly to the HTML document during page load.

```
document.write("Hello, World!");
```

Q5. Explain prompt() Functions in JavaScript

prompt() displays a dialog box that prompts the user to input text.

```
let name = prompt("Enter your name:");  
console.log(name);
```

Q6. Data Types in JavaScript

1. String
 2. Number
 3. Boolean
 4. Object
 5. Undefined
 6. Null
 7. Symbol
 8. BigInt
-

Q7. typeof Operator in JavaScript

The typeof operator determines the type of a variable or value.

```
console.log(typeof "Hello"); // Output: string
```

```
console.log(typeof 42); // Output: number
```

Q8. Difference Between Programming, Scripting, and Markup Languages

Programming	Scripting	Markup
General-purpose.	Task-oriented (e.g., JS).	Structure content (HTML).
Compiled or interpreted.	Interpreted at runtime.	Declarative syntax.

Q9. Difference Between == and === Operators

1. ==: Compares values after type conversion.

```
5 == "5"; // true
```

2. ===: Compares values without type conversion.

```
5 === "5"; // false
```

Q10. NaN Property in JavaScript

NaN (Not-a-Number) indicates an invalid number result.

```
console.log(0 / 0); // NaN
```

Q11. Explain the this Keyword

Refers to the current execution context.

```
const obj = { name: "Alice", greet() { return this.name; } };
```

```
console.log(obj.greet()); // Alice
```

Q12. Why Use Callbacks?

Callbacks allow asynchronous operations by executing a function after an operation is completed.

Q13. What is BOM in JS?

The **Browser Object Model (BOM)** provides methods and properties for interacting with the browser.

Q14. Define Anonymous Function

A function without a name, often used in callbacks.

```
let greet = function () { console.log("Hello!"); };  
greet();
```

Q15. Convert a String to Lowercase in JavaScript

Use the `toLowerCase()` method.

```
console.log("HELLO".toLowerCase()); // hello
```

Q16. Write HTML Code Dynamically Using JavaScript

```
document.body.innerHTML = "<h1>Dynamic HTML</h1>";
```

Q17. Output of `10+20+"30"` in JavaScript

Output: "3030"

Explanation: `10 + 20` is added, then concatenated with `"30"`.

Q18. Different Pop-Up Boxes in JavaScript

1. `alert()`
 2. `prompt()`
 3. `confirm()`
-

Q19. Use of Math Object in JavaScript

Performs mathematical operations.

```
console.log(Math.sqrt(16)); // 4
```

Q20. Undeclared vs Undefined Variables

- **Undeclared:** Variable that is not declared.

- **Undefined:** Variable declared but not assigned a value.
-

LEVEL B - INTERMEDIATE QUESTIONS (5 MARKS EACH)

Q21. Explain Hoisting in JavaScript with Suitable Code

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope during execution. However, only the declaration is hoisted, not the initialization.

`console.log(a);` // Output: undefined (declaration is hoisted, but not the initialization)

`var a = 5;`

`foo();` // Output: Hello (function declarations are fully hoisted)

`function foo() {`

`console.log("Hello");`

`}`

Variables declared with `let` and `const` are hoisted but remain in a temporal dead zone until initialized.

`console.log(b);` // ReferenceError

`let b = 10;`

Q22. Explain Implicit Type Coercion in JavaScript

Implicit type coercion occurs when JavaScript automatically converts one data type to another during operations.

`console.log("5" - 3);` // Output: 2 (string "5" is converted to number)

`console.log("5" + 3);` // Output: "53" (number 3 is converted to string)

`console.log(true + 2);` // Output: 3 (true is converted to 1)

It can lead to unexpected results, so explicit type conversions are often preferred.

Q23. Purpose of this in JavaScript

The `this` keyword refers to the execution context, which varies based on how the function is invoked:

- **Global Scope:** Refers to the global object (window in browsers).

- **Object Method:** Refers to the object calling the method.
- **Arrow Functions:** Lexically bind this from their surrounding scope.

Example:

```
function regular() {  
  console.log(this); // Refers to the caller object or global object  
}
```

```
const obj = {  
  method() {  
    console.log(this); // Refers to obj  
  }  
};
```

```
regular(); // Global object  
obj.method(); // obj
```

Q24. What is DOM? Explain with a Suitable Example

The **Document Object Model (DOM)** is a tree-like structure representing the HTML document as objects. It allows JavaScript to interact with and manipulate the webpage.

Example:

```
<p id="demo">Hello, DOM!</p>
```

JavaScript:

javascript

Copy code

```
document.getElementById("demo").innerHTML = "Hello, World!";
```

Q25. What are Arrow Functions? Explain with Suitable Code

Arrow functions are a concise syntax for writing functions, introduced in ES6. They do not have their own this context, instead inheriting it lexically.

```
const add = (a, b) => a + b;
```

```
console.log(add(5, 3)); // Output: 8
```

```
const obj = {  
  value: 10,  
  arrow: () => console.log(this.value), // Inherits `this` from outer context  
  regular() {  
    console.log(this.value); // Refers to `obj`  
  }  
};
```

```
obj.arrow(); // Undefined (arrow function doesn't bind `this`)
```

```
obj.regular(); // 10
```

Q26. Rest Parameter and Spread Operator

Rest Parameter: Collects multiple arguments into an array.

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}
```

```
console.log(sum(1, 2, 3)); // Output: 6
```

Spread Operator: Expands an array or object into individual elements.

```
const arr = [1, 2, 3];  
console.log(...arr); // Output: 1 2 3
```

Q27. Difference Between splice() and slice()

splice()

Modifies the original array.

Can add or remove elements.

Syntax: array.splice(start, deleteCount)

Example:

slice()

Does not modify the original array.

Returns a shallow copy of a portion.

Syntax: array.slice(start, end)

```
let arr = [1, 2, 3, 4];  
arr.splice(1, 2); // Modifies arr: [1, 4]  
let copy = arr.slice(0, 1); // arr: [1, 4]; copy: [1]
```

Q28. What is a Callback Function in JavaScript?

A callback is a function passed as an argument to another function, executed after the completion of that function.

Example:

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}  
  
function bye() {  
  console.log("Goodbye!");  
}  
  
greet("Sufiyan", bye);  
  
// Output:  
// Hello Sufiyan  
// Goodbye!
```

Q29. Explain Array and Object Destructuring with Proper Code

Array Destructuring allows unpacking values from arrays into distinct variables.

Object Destructuring extracts properties from objects into variables.

Example:

```
// Array Destructuring  
const numbers = [1, 2, 3];  
const [a, b, c] = numbers;  
console.log(a, b, c); // Output: 1 2 3  
  
// Skipping elements
```

```
const [x, y] = numbers;
console.log(x, y); // Output: 1 3
```

```
// Object Destructuring
```

```
const book = { title: "JavaScript", author: "Sufiyan", price: 500 };
const { title, price } = book;
console.log(title, price); // Output: JavaScript 500
```

```
// Renaming variables
```

```
const { title: bookTitle } = book;
console.log(bookTitle); // Output: JavaScript
```

Q30. Different Ways to Select an Element in JavaScript

1. **getElementById**: Selects an element by its ID.

```
const element = document.getElementById("myId");
console.log(element);
```

2. **getElementsByClassName**: Selects elements by class name.

```
const elements = document.getElementsByClassName("myClass");
console.log(elements[0]);
```

3. **getElementsByTagName**: Selects elements by tag name.

```
const elements = document.getElementsByTagName("div");
console.log(elements[0]);
```

4. **querySelector**: Selects the first matching element.

```
const element = document.querySelector(".myClass");
console.log(element);
```

5. **querySelectorAll**: Selects all matching elements as a NodeList.

```
const elements = document.querySelectorAll(".myClass");
console.log(elements[0]);
```

Q31. JavaScript Function to Capitalize First Letter of Each Word

```
function capitalizeWords(str) {  
    return str  
        .split(" ")  
        .map(word => word.charAt(0).toUpperCase() + word.slice(1))  
        .join(" ");  
}  
  
console.log(capitalizeWords("hello world from sufiyan"));  
  
// Output: Hello World From Sufiyan
```

Q32. Difference Between Function Declaration and Function Expression**Function Declaration**

Named function.

Hoisted to the top of the scope.

Syntax: function foo() {}

Function Expression

Can be named or anonymous.

Not hoisted; must be declared first.

Syntax: const foo = function() {}

Example:

// Function Declaration

```
function greet() {  
    console.log("Hello");  
}
```

// Function Expression

```
const greetExpr = function () {  
    console.log("Hi");  
};
```

greet(); // Hello

greetExpr(); // Hi

Q33. Ways to Register Events to a Button Element**1. Inline HTML**

```
<button onclick="alert('Button Clicked!')">Click Me</button>
```

2. Using JavaScript DOM

```
const btn = document.getElementById("myButton");
```

```
btn.onclick = () => alert("Clicked!");
```

3. addEventListener Method

```
const btn = document.getElementById("myButton");
```

```
btn.addEventListener("click", () => alert("Event Listener Clicked!"));
```

Q34. JavaScript Modules with Import and Export**Exporting:**

```
// math.js
```

```
export const add = (a, b) => a + b;
```

```
export const subtract = (a, b) => a - b;
```

Importing:

```
// main.js
```

```
import { add, subtract } from "./math.js";
```

```
console.log(add(5, 3)); // Output: 8
```

```
console.log(subtract(5, 3)); // Output: 2
```

Q35. Create a "Book" Object with Properties and Methods

```
const book = {
```

```
  title: "JavaScript Basics",
```

```
  author: "Sufiyan Khan",
```

```
  price: 1000,
```

```
  getDetails() {
```

```
    return `${this.title} by ${this.author}, priced at $${this.price}`;
```

```
    },

    setPrice(newPrice) {
        this.price = newPrice;
    },

    applyDiscount(discount) {
        this.price -= this.price * (discount / 100);
    }
};

book.applyDiscount(10);
console.log(book.getDetails()); // Output: JavaScript Basics by Sufiyan Khan, priced at $900
```

Q36. Map() and Filter() Functions with ES6 Module Example

Map Function: Transforms elements in an array.

Filter Function: Filters elements based on a condition.

Example:

```
const numbers = [1, 2, 3, 4, 5];
const squared = numbers.map(n => n ** 2);
console.log(squared); // Output: [1, 4, 9, 16, 25]
const evenNumbers = numbers.filter(n => n % 2 === 0);
console.log(evenNumbers); // Output: [2, 4]
```

ES6 Module for Factorial:

```
// factorial.js
export const factorial = n => (n <= 1 ? 1 : n * factorial(n - 1));
```

Import and Use:

```
// main.js
import { factorial } from './factorial.js';
console.log(factorial(5)); // Output: 120
```